Microelectronics Journal 157 (2025) 106579

Contents lists available at ScienceDirect

Microelectronics Journal

journal homepage: www.elsevier.com/locate/mejo

Exact and approximate Radix-4 recoding multipliers for high-efficiency computation

Xinyu Zhu, Hongge Li *, Yinjie Song

School of Electronic Information Engineering, Beihang University, Beijing, 100191, China

ARTICLE INFO

Index Terms:
Exact and approximate multipliers
Booth recoding
Radix-4
Energy efficiency

ABSTRACT

—In this paper, novel exact and approximate encoding methods are proposed to improve the computation efficiency of the Booth multiplier. The proposed encoding methods realize the transformation from non-zero to zero encodings of the radix-4 Booth algorithm, which increases the number of zero encodings. The proposed exact and approximate recoding multiplication algorithms reduce the computing operation of the radix-4 Booth multiplier by skipping zero encodings. Therefore, the multiplier computation time and energy consumption are reduced. Based on these algorithms, an exact radix-4 recoding multiplier (ER4RM) and two approximate radix-4 recoding multipliers (AR4RM1 and AR4RM2) with different accuracies are proposed. The multipliers were synthesized using a standard CMOS 28-nm process and verified using several different digit sets, and the energies are 0.25, 0.24 and 0.23 pJ based on typical 16-bit widths for ER4RM, AR4RM1 and AR4RM2, respectively. The implemented results show a significant energy and energy-area efficiency advantage over existing multiplier designs, with slight accuracy loss.

1. Introduction

High-efficiency multipliers are widely used in arithmetic units of processors, especially low-power RISC processors, and machine learning (ML) [1–4]. Many algorithms and architectures have been proposed for designing advanced low-power multipliers [5–8]. Modified Booth encoding is currently the most effective algorithm for improving the performance of multipliers and reducing their area and energy overhead [9–11]. However, energy consumption is constantly increasing because of the rising complexity of arithmetic circuits [12–14]. As a key component of these arithmetic units, it is of great significance to effectively reduce the cost and energy consumption of multipliers.

Currently, many methods have been proposed to reduce the cost and energy consumption of exact multipliers, such as optimizing the encoding algorithm and reducing the redundant calculation [15–18], etc. Tsoumanis et al. [15] presented a non-redundant radix-4 signed-digit encoding technique to improve power efficiency. In Ref. [16], the parallel radix-4 Booth multiplier reduces area and energy by removing additional compensation circuits and reducing partial accumulation. In addition, Ellaithy et al. [17] presented a dual-channel serial computing method that reduces the hardware complexity of the multiplier. For those special multipliers, such as squares, Chen et al. [18] proposed an accurate squarer based on a radix-8 Booth-folding square encoding algorithm, which reduces the number of partial products and thus reduces cost and energy consumption.

Many approximate multipliers have been proposed in recent years to improve speed and reduce power [19–21]. They are used in error-tolerant applications, further reducing energy consumption [22–28]. In Ref. [22], Haider et al. proposed a novel recoding method that combines Booth encoding and power-of-two quantization to reduce the size of DNN weights, enabling low-energy computations with minimal accuracy loss. Venkatachalam et al. [23] proposed three radix-4 Booth approximate multipliers, which encode ±2 in radix-4 Booth encoding as ±1, therefore simplifying the encoding and calculation circuits and reducing energy consumption. The hybrid low radix multiplier (HLR-BM) is presented in Ref. [24], it approximates the odd multiples ±3 of radix-8 Booth encoding to ±2 and ± 4, which reduces the cost of the circuit. In addition, an approximate compressor was introduced in Ref. [25], which optimizes the critical path to reduce energy. Du et al. [26] proposed a one/zero detector-based approximate multiplier for signed multiplication, which dynamically truncates insignificant encoding. To improve energy efficiency and accuracy, Park [27] simplified the Booth encoder and compensated for encoding errors. Zhang et al. [28] proposed approximate radix-4 Booth multipliers based

* Corresponding author.
E-mail addresses: zxyuser@buaa.edu.cn (X. Zhu), honggeli@buaa.edu.cn (H. Li), yinjiesong@buaa.edu.cn (Y. Song).

https://doi.org/10.1016/j.mejo.2025.106579
Received 27 September 2024; Received in revised form 2 December 2024; Accepted 17 January 2025
Available online 23 January 2025

on majority logic to reduce power and area in emerging nanotechnologies. The above studies show that optimization of Booth encoding by approximation can effectively reduce energy consumption. Moreover, in the radix-4 Booth multiplier, the zero encodings are special encodings, which produces partial products that do not affect the result, and the zero encoding calculations can be skipped by a special design, thus improving the computation efficiency. Therefore, we propose novel exact and approximate recoding methods and radix-4 recoding multipliers, which recode some non-zero encodings to zero encodings and skip zero encoding computations, thus reducing energy consumption. The main contributions of this study can be summarized as follows.

1) The novel exact and approximate recoding methods are proposed, which convert non-zero to zero encodings of the radix-4 Booth algorithm to increase zero encodings and reduce the number of computations.
2) The exact and approximate multipliers with radix-4 recoding are proposed, which detect zero encodings and then skip the zero partial products and addition calculations to reduce energy consumption.
3) The synthesis results of the standard CMOS process are estimated by different test sets to evaluate the performance of the proposed exact and approximate multipliers. In addition, the hardware and accuracy tradeoffs of approximate multipliers are analyzed and compared.

This paper is organized as follows. Section II reviews the radix-4 Booth encoding and multiplication. Section III presents the novel exact and approximate recoding methods. Section IV introduces the exact and approximate radix-4 recoding multipliers. Section V presents the implementation results and accuracy comparison of the proposed multiplier with other multipliers. Section VI concludes this paper.

2. Radix-4 boothencoding and multiplication

Booth encoding is used to solve the signed number multiplication of two complement numbers [9]. It has been revised to the modified Booth encoding or radix-4 Booth encoding [11,29]. The radix-4 Booth encoding divides multiplier $A(a_{n-1}a_{n-2}\ \ldots\ a_2a_1a_0)$ into overlapping groups of three contiguous bits. These groups were encoded and decoded using multiplicand B to generate a partial product [30]. the n-bit multiplier A of the radix-4 Booth encoding can be expressed as

$$\begin{aligned} A &= -a_{n-1}2^{n-1} + \sum_{i=0}^{n-2} a_i 2^i \\ &= \sum_{i=0}^{(n-2)/2} (-2a_{2i+1} + a_{2i} + a_{2i-1})2^{2i} \\ &= \sum_{i=0}^{(n-2)/2} M_i 2^{2i}, \end{aligned} \tag{1}$$

where M_i is a coefficient with radix-4 Booth encoding, for which the digit set is $\{\overline{2}, \overline{1}, 0, 1, 2\}$. The partial products produced by this coefficient can be shifted and accumulated to complete the multiplication. Radix-4 Booth serial multiplication considers several digits of multiplier A simultaneously and is computed in $(n+1)/2$ partitions, which can be utilized in energy-efficiency designs, such as low-power RISC-V processors.

First, the n-bit multiplier A is encoded by radix-4 Booth encoding. Then, the encoded digit is multiplied by each bit of multiplicand B to obtain the partial product. Finally, the partial product is added to the previous partial product and shifted. This calculation is expressed as follows:

$$\begin{aligned} &P^{(j+1)} = 2^{-2}\left(P^{(j)} + 2^n\left(-2a_{2j+1} + a_{2j} + a_{2j-1}\right)B\right), \\ &P^{(0)} = 0, j = 0, 1, \ldots, (n-1)/2 \end{aligned} \tag{2}$$

The multiplication is represented as an algorithm in Algorithm 1, where "ASR" is the arithmetic shift right function.

Algorithm 1 Radix-4 Booth Multiplication

Input: *A: Multiplier*; *B*: *Multiplicand*
Output: *P: Product*
1: *P[n-1:0] = A, a_-1= 0;*
2: **for** *i from 0 to (n-1)/2* **do**
3: *M_i = -2a_{2i+1}+a_{2i} + a_{2i-1};//R-4 Booth encoding (M_i)*
4: *P = ASR(P,2);//Arithmetic shift right*
5: *PP = M_i×B;//Partial product*
6: *P[2n-1:n] = P[2n-1:n]+PP;*
7: **end for**
8: **return** *P*

3. Exact and approximate Radix-4 booth recoding methods

In this section, we present the novel exact and approximate recoding methods. These methods convert some of the neighboring non-zero encodings in the radix-4 Booth encodings into new encodings containing zeros, thus increasing the proportion of zero encodings. In this way, when the multiplier skips the computation of zero encodings, the computational operations are further reduced, which improves the performance of the multiplier and reduces the energy consumption. According to (1), the radix-4 Booth encoding can be expressed as follows:

$$\begin{aligned} A &= \sum_{i=0}^{(n-2)/2} M_i 2^{2i} = \sum_{i=0}^{(n-4)/4} (4M_{2i+1} + M_{2i})2^{4i} \\ &= \sum_{i=0}^{(n-4)/4} (4(M_{2i+1} - M_{2i+1}) + (M_{2i} + 4M_{2i+1}))2^{4i} \end{aligned} \tag{3}$$

The new coefficient K_i can be used for (3), which is expressed as follows:

$$K_{2i+1} = M_{2i+1} - M_{2i+1} \tag{4}$$

$$K_{2i} = M_{2i} + 4M_{2i+1} \tag{5}$$

According to (3), (4) and (5), the novel encoding K_{2i+1} and K_{2i} can replace the Booth encoding M_{2i+1} and M_{2i}; thus, multiplier A can be described as follows:

$$A = \sum_{i=0}^{(n-4)/4} (4K_{2i+1} + K_{2i})2^{4i} = \sum_{i=0}^{(n-2)/2} K_i 2^{2i} \tag{6}$$

The digit set of K_i is the same as M_i, which is$\{\overline{2}, \overline{1}, 0, 1, 2\}$. According to (4) and (5), there are two exact non-zero-transformable cases:

1) If $(M_{i+1}\ M_i) = (\overline{1}\ 2)$, then $(K_{i+1}\ K_i) = (0\ \overline{2})$.
2) If $(M_{i+1}\ M_i) = (1\ \overline{2})$, then $(K_{i+1}\ K_i) = (0\ 2)$.

The two encoding conversion methods mentioned above ensure the correctness before and after the conversion of radix-4 Booth encoding. However, due to the limitations of the digit set of K_i, other non-zero M_i cannot be converted successfully. Based on the above analysis, we proposed an exact recoding method, which can generate more zero encodings by recoding, including both the zero encodings in the Booth algorithm and two zero encodings discussed earlier. Compared with the

Table 1
The skippable exact and approximate zero encodings by recoding.

Type	$a_{2i+3}a_{2i+2}a_{2i+1}a_{2i}a_{2i-1}$	$K_{i+1}K_i$	$a_{2i+3}a_{2i+2}a_{2i+1}$ $a_{2i}a_{2i-1}$	$K_{i+1}K_i$
Exact	00000	00	10000	$\bar{2}0$
	00001	01	10111	$\bar{1}0$
	00010	01	11000	$\bar{1}0$
	00011	02	11100	$0\bar{2}$
	00100	$1\bar{2}\rightarrow 02$	11011	$\bar{1}2\rightarrow 0\bar{2}$
	00111	10	11101	$0\bar{1}$
	01000	10	11110	$0\bar{1}$
	01111	20	11111	00
Approximate	00101	$1\bar{1}\rightarrow 02$	11001	$\bar{1}1\rightarrow 0\bar{2}$
	00110	$1\bar{1}\rightarrow 02$	11010	$\bar{1}1\rightarrow 0\bar{2}$

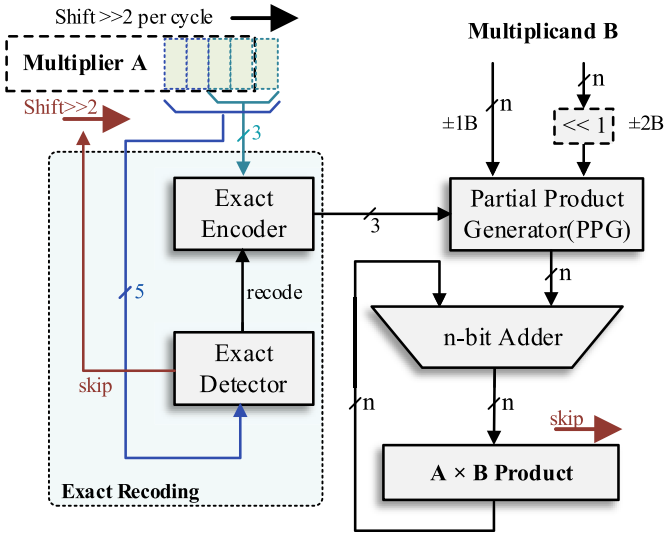

Fig. 1. Exact radix-4 recoding multiplier.

traditional radix-4 Booth encoding, the proposed method produces more zero encodings, and the multiplier skips zero encodings, thereby reducing the number of operations and improving computational efficiency. The exact recoding coefficient EK_i is denoted as follows:

$$EK_{i+1}=\begin{cases}0, if\ a_{2i+3}a_{2i+2}a_{2i+1}a_{2i}a_{2i-1}=\{00100,11011\};\\ -2a_{2i+3}+a_{2i+2}+a_{2i+1}, otherwise.\end{cases} \tag{7}$$

$$EK_i=\begin{cases}2a_{2i+1}-a_{2i}-a_{2i-1i},\\ if\ a_{2i+3}a_{2i+2}a_{2i+1}a_{2i}a_{2i-1}=\{00100,11011\};\\ -2a_{2i+1}+a_{2i}+a_{2i-1}, otherwise.\end{cases} \tag{8}$$

To recode more zero encodings and ensure less loss of accuracy in the product, we proposed an approximate recoding method. According to (4), (5) and the range of the digit set of K_i, there are two approximate non-zero-transformable cases.

1) If $(M_{i+1}\ M_i) = (1\ \bar{1})$, then $(K_{i+1}\ K_i) = (0\ 3)$, after approximate recoding $(K_{i+1}\ K_i) = (0\ 2)$.
2) If $(M_{i+1}\ M_i) = (\bar{1}\ 1)$, then $(K_{i+1}\ K_i) = (0\ \bar{3})$, after approximate recoding $(K_{i+1}\ K_i) = (0\ \bar{2})$.

Algorithm 2 Novel Exact Recoding Multiplication

Input: *A: Multiplier; B: Multiplicand*
Output: *P: Product*
1: $P[n\text{-}1{:}0] = A,\ a_{-1} = 0,\ Skip_0 = 0;$
2: **for** *i from 0 to (n-1)/2* **do**

(*continued on next column*)

(*continued*)

Algorithm 2 Novel Exact Recoding Multiplication

3: $M_i = -2a_{2i+1}+a_{2i}+a_{2i-1}$;//*R-4 Booth encoding*
4: $EK_i = (a_{2i+3}\ \&a_{2i+2}\&\sim a_{2i+1}\&a_{2i}\&a_{2i-1})$
5: $\quad\mid(\sim a_{2i+3}\&\sim a_{2i+2}\&a_{2i+1}\&\sim a_{2i}\&\sim a_{2i-1})$;
6://*Exact recoding*
7: $P = ASR(P,2)$;//*Arithmetic shift right*
8: **if** $(Skip_i=0)$ **then**//*Non-zero, non-skip*
9: **if** $(EK_i=1)$ **then**
10: $PP = -M_i\times B$;
11: **else**
12: $PP = M_i\times B$;
13: **end if**
14: $P[2n\text{-}1{:}n] = P[2n\text{-}1{:}n]+PP$;
15: **end if**
16: $Skip_{i+1} = (M_{i+1}=0)\mid EK_i$;
17: **end for**
18: **return** *P*

Algorithm 3 Novel Approximate Recoding Multiplication

Input: *A: Multiplier; B: Multiplicand*
Output: *P: Product*
1: $P[n\text{-}1{:}0] = A,\ a_{-1} = 0,\ Skip_0 = 0;$
2: **for** *i from 0 to (n-1)/2* **do**
3: $M_i = -2a_{2i+1}+a_{2i}+a_{2i-1}$;//*R-4 Booth encoding*
4: $AK1_i = \sim a_{2i+3}\ \&\sim a_{2i+2}\&a_{2i+1}\&\sim(a_{2i}\ \&a_{2i-1})$;
5: $AK2_i = a_{2i+3}\ \&a_{2i+2}\&\sim a_{2i+1}\&(a_{2i}\mid a_{2i-1})$;
6://*Approximate recoding*
7: $P = ASR(P,2)$;//*Arithmetic shift right*
8: **if** $(Skip_i=0)$ **then**//*Non-zero, non-skip*
9: **if** $(AK1_i=1)$ **then**
10: $PP = 2B$;
11: **else if** $(AK2_i=1)$ **then**
12: $PP = -2B$;
13: **else**
14: $PP = M_i\times B$;
15: **end if**
16: $P[2n\text{-}1{:}n] = P[2n\text{-}1{:}n]+PP$;
17: **end if**
18: $Skip_{i+1} = (M_{i+1}=0)\mid AK1_i \mid AK2_i$;
19: **end for**
20: **return** *P*

The coefficients K_{i+1} and K_i after approximate recoding are 0 and ± 2, respectively, which are the same as the exact recoding coefficients. Moreover, the error distance between this approximate coding and exact coding is ±1 at 5-bit. Therefore, the approximate recoding coefficient AK_i is denoted as follows:

$$AK_{i+1}=\begin{cases}0, if\ a_{2i+3}a_{2i+2}a_{2i+1}a_{2i}a_{2i-1}=\{00100,\\ 00101,00110,11011,11010,11001\};\\ -2a_{2i+3}+a_{2i+2}+a_{2i+1}, otherwise.\end{cases} \tag{9}$$

$$AK_i=\begin{cases}2, if\ a_{2i+3}a_{2i+2}a_{2i+1}a_{2i}a_{2i-1}=\{00100,00101,00110\};\\ -2, if\ a_{2i+3}a_{2i+2}a_{2i+1}a_{2i}a_{2i-1}=\{11011,11010,11001\};\\ -2a_{2i+1}+a_{2i}+a_{2i-1}, otherwise.\end{cases} \tag{10}$$

Skippable zero encodings are defined as zero codes in the radix-4 recoding methods that can be skipped during multiplication. The skippable exact and approximate zero encodings generated by the recoding method are presented in Table 1, which provides all cases of skippable zero encodings in the 5-bit multiplier *A*. Table 1 also includes two exact encodings and four approximate encodings. They are converted from non-zero to zero, which increases the number of zero encodings.

The novel exact recoding multiplication method is presented in Algorithm 2, which can skip all the zero encodings in the exact type of Table 1. The $Skip_i$ represents skipping partial product calculations, EK_i represents the exact recoding method. In the first loop, $Skip_i$ is initialized to zero ($Skip_0 = 0$), then, the partial product is calculated. If the next Booth encoding M_{i+1} is encoded as zero or the exact recoding EK_i is one ("00100" or "11011"), then $Skip_{i+1}$ is set to one, which means skipping

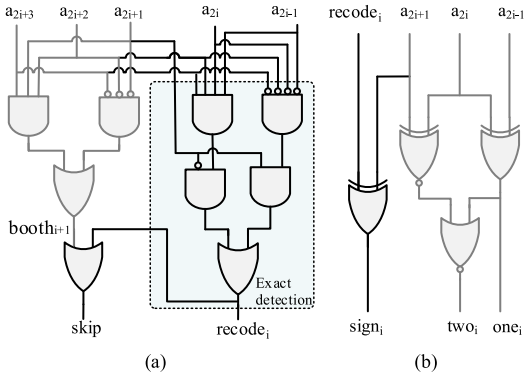

Fig. 2. Exact recoding, (a) exact detector, (b) exact encoder.

Time	Radix-2 multiplier A >>2	Radix-4 encoding Booth	Radix-4 encoding Novel	Execute Partial Product	Execute Skip
t0	011011000**10000**	$2\bar{1}2\bar{2}1\bar{2}0$	$2\bar{1}\bar{2}21\bar{2}0$	0B * 2^0	1
t1	0110110**00100**	$2\bar{1}2\bar{2}1\bar{2}$	$2\bar{1}2\bar{2}02$	2B * 2^2	0
t2	01101**10001**	$2\bar{1}2\bar{2}1$	$2\bar{1}2\bar{2}0$	0B * 2^4	1
t3	011**01100**	$2\bar{1}2\bar{2}$	$2\bar{1}2\bar{2}$	-2B * 2^6	0
t4	0**11011**	$2\bar{1}2$	$20\bar{2}$	-2B* 2^8	0
t5	**00110**	$\bar{2}1$	$\bar{2}0$	0B * 2^{10}	1
t6	**00001**	$\bar{2}$	$\bar{2}$	-2B * 2^{12}	0

Shift right 2-bit | Encoder bits | zero encoding | $2\bar{}$ represent -2

Fig. 3. Procedure for the proposed exact multiplier.

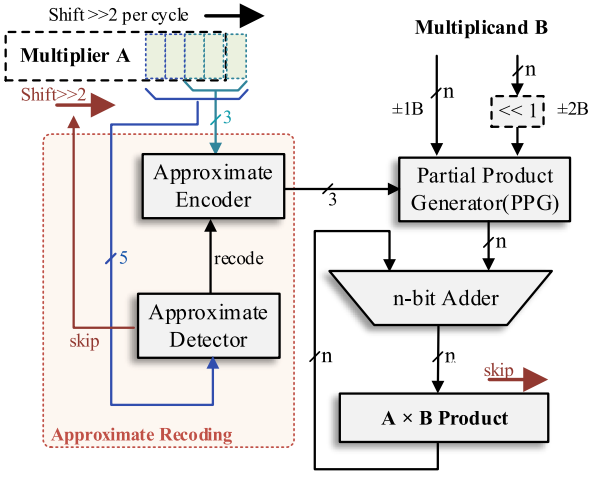

Fig. 4. Approximate radix-4 recoding multiplier.

the partial product and addition calculation in the next cycle. According to (7) and (8), when EK_i is one, K_{i+1} and K_i are set to zero and $-M_i$, otherwise, they are set to M_{i+1} and M_i, respectively.

The novel approximate recoding multiplication method is presented in Algorithm 3, which skips all the zero encodings in Table 1. It detects approximate encodings and generates approximate partial products according to (9) and (10). $AK1_i$ represents the approximate codes "00100", "00101" and "00110", and $AK2_i$ represents the approximate codes "11011", "11001" and "11010". When $AK1_i$ or $AK2_i$ are valid ($AK1_i|AK2_i = 1$) or the next Booth encoding M_{i+1} is zero, the partial product calculation can be skipped in the next loop.

4. Exact and approximate Radix-4 recoding multipliers

4.1. Exact multipliers

4.1.1. Exact multipliers architecture

An exact radix-4 recoding multiplier, which is an extension of the radix-4 Booth serial multiplier, is proposed. The proposed n-bit multiplier is illustrated in Fig. 1, where "n" represents the bit width of the exact multiplier. It consists of several components: multiplier *A*, multiplicand *B*, partial product generator (PPG), adder, product, and exact recoding module (exact detector and exact encoder). The exact recoding module detects the least significant 5-bit of multiplier *A* and recodes them according to the exact recoding methods in (7) and (8). The partial product is generated by decoding multiplicand *B* with the recoding result, where 2B is generated by shifting the multiplicand 1-bit to the left. A partial product is then added to the product. The product and multiplier *A* are shifted to the right by two bits per cycle, and the final product is obtained until the highest bit of multiplier *A* is processed. When the skippable exact code in Table 1 is detected, multiplier *A* and the product are shifted 2-bit to the right to skip the cycle.

4.1.2. Exact recoding

The exact recoding module includes an exact detector and exact encoder. The detector circuit that recognizes skippable exact zero encodings in multiplier *A* is shown in Fig. 2(a). According to Table 1, the detector generates a "skip" signal by detecting the 5-bit of multiplier *A*. This signal is used to indicate the skipping of the product and addition operations. During the first cycle, the detection includes the least significant 4-bit of multiplier *A* and zero. The detector is divided into two parts, namely, $booth_{i+1}$, representing zeros in the radix-4 Booth encodings of $a_{2i+3}a_{2i+2}a_{2i+1}$, and $recode_i$, representing zeros in the recoding of $a_{2i+3}a_{2i+2}a_{2i+1}a_{2i}a_{2i-1}$. The signal $recode_i$ and skip can be expressed as follows:

$$recode_i = a_{2i+3}a_{2i+2}\overline{a_{2i+1}}a_{2i}a_{2i-1} + \overline{a_{2i+3}a_{2i+2}}a_{2i+1}\overline{a_{2i}a_{2i-1}} \tag{11}$$

$$skip = a_{2i+3}a_{2i+2}a_{2i+1} + \overline{a_{2i+3}a_{2i+2}a_{2i+1}} + recode_i \tag{12}$$

The exact encoder is shown in Fig. 2(b), which completes the radix-4 recoding of the multiplier in (7) and (8). The signal $sign_i$ denotes the recoded symbol, and one_i and two_i denote the recoded absolute values. According to (7) and (8), the encoder can be expressed as:

$$sign_i = recode_i \oplus a_{2i+1} \tag{13}$$

$$one_i = a_{2i-1} \oplus a_{2i} \tag{14}$$

$$\begin{aligned} two_i &= \overline{a_{2i+1}}a_{2i}a_{2i-1} + a_{2i+1}\overline{a_{2i}a_{2i-1}} \\ &= \overline{a_{2i-1} \oplus a_{2i}} \cdot (a_{2i+1} \oplus a_{2i}) \\ &= \overline{a_{2i-1} \oplus a_{2i} + \overline{a_{2i+1} \oplus a_{2i}}}\ . \end{aligned} \tag{15}$$

4.1.3. Recoding procedure of the exact multiplier

Fig. 3 illustrates the procedure for the proposed exact multiplier. Multiplier *A* and multiplicand *B* are both 2's complement numbers, the a_{-1} of *A* is zero in the first cycle. "Booth" denotes traditional radix-4 Booth encoding, and "Novel" denotes the proposed novel recoding. The least significant 5-bit multiplier *A* was detected per cycle. In the first cycle, "10000" is determined to be a skippable encoding according to Table 1, which produces a partial product of "0B; " thus, this cycle can be skipped. The next 5-bit encoding, "00100," is also a skippable encoding, which allows skipping the calculation in the next loop. Therefore, the partial product is "2B" in the current cycle, and the partial product "0B" is skipped in the next cycle. Then, the next 5-bit encoding, "00110," represents a non-zero encoding, and thus cannot be skipped. Therefore, the multiplier generates the partial product "-2B" and completes the addition calculation in this cycle. When the loop detects the last cycle, empty bits are complemented by zeros. Once all the bits of multiplier *A*

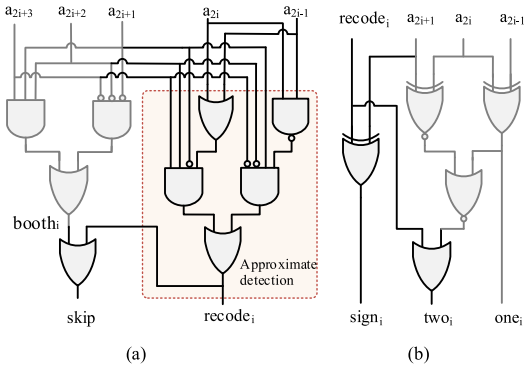

Fig. 5. Approximate Recoding,(a) approximate detector, (b) approximate encoder.

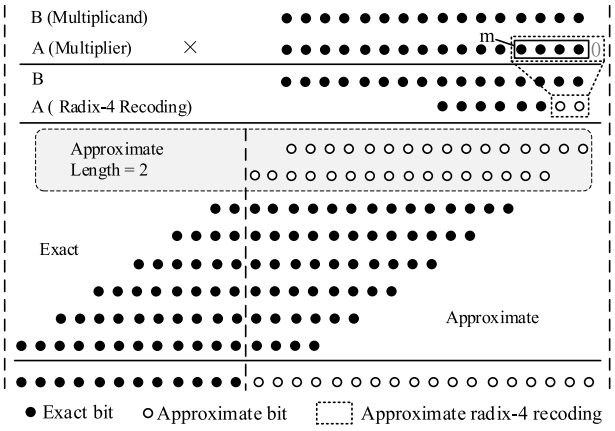

Fig. 6. The dot diagram of a 16-bit approximate recoding radix-4 Booth multiplier with the approximate length of 2.

are processed, the final result is generated.

4.2. Approximate multipliers

4.2.1. Approximate multipliers architecture

The proposed n-bit approximate radix-4 recoding multiplier is shown in Fig. 4, where "n" represents the bit width of the approximate multiplier. It is similar to the exact multiplier and consists of several components: multiplier *A*, multiplicand *B*, partial product generator (PPG), adder, product, and approximate recoding module (approximate detector and approximate encoder). The approximate recoding module detects the least significant 5-bit of multiplier *A* and recodes them according to the approximate recoding methods in (9) and (10). The generation of partial products and products is the same as for the exact multiplier. However, unlike the exact multiplier, the approximate multiplier detects all skippable zero encodings in Table 1, including exact and approximate zero encodings. Therefore, the approximate multiplier can further reduce the computations.

4.2.2. Approximate recoding

The approximate recoding module includes an approximate detector and approximate encoder. The approximate detector is shown in Fig. 5 (a), which recognizes all skippable zero encodings. According to Table 1, (9) and (10), the approximate multiplier needs to detect 5-bit of multiplier A to generate the "recode$_i$" signal. As shown in the orange area in Fig. 5(a), the approximate detector consists of a NAND gate, two OR gates, and two four-input AND gates. The signal "skip" is the same as the exact detector in (12). The signal "recode$_i$" can be expressed as follows:

$$recode_i = \overline{a_{2i+3}a_{2i+2}}a_{2i+1}\overline{(a_{2i}a_{2i-1})} + a_{2i+3}a_{2i+2}\overline{a_{2i+1}}(a_{2i} + a_{2i-1}) \tag{16}$$

The approximate encoder is shown in Fig. 5(b), which completes the approximate recoding of the multiplier in (9) and (10). Compared to the exact encoder, the approximate encoder adds an OR gate to generate "two$_i$" signal. The signal "sign$_i$" and "one$_i$" are the same as the exact detector in (13) and (14). According to (9) and (10), the encoder can be expressed as:

$$\begin{aligned} two_i &= recode_i + \overline{a_{2i+1}}a_{2i}a_{2i-1} + a_{2i+1}\overline{a_{2i}a_{2i-1}} \\ &= recode_i + \overline{a_{2i-1} \oplus a_{2i} + \overline{a_{2i+1} \oplus a_{2i}}} \end{aligned} \tag{17}$$

4.2.3. Approximate length

Although all bits of the multiplier A can be encoded using the approximate recoding method, it leads to poor accuracy performance. Therefore, in the proposed multiplier, the approximate recoding method is employed in the least significant "m" bits of A, and exact recoding is employed for the other bits. Moreover, we define the approximation length, which represents the number of approximate bits in the radix-4 recoding of multiplier A. Thus, the approximation length is equal to "m/2", where "m" represents the number of bits used in the approximate recoding of the multiplier A. The approximation length also corresponds to the number of approximate partial products in the partial product matrix.

In this paper, we designed two approximate radix-4 recoding multipliers AR4RM1 and AR4RM2 with different approximate lengths. Specifically, the approximate lengths of AR4RM1 and AR4RM2 are set to the typical values of 2 and 4, respectively. Fig. 6 shows a dot diagram of a 16-bit recoding multiplier with an approximate length of 2. The least significant 4 bits of A and zero are encoded using the approximate recoding method, whereas the most significant 12 bits of A are encoded

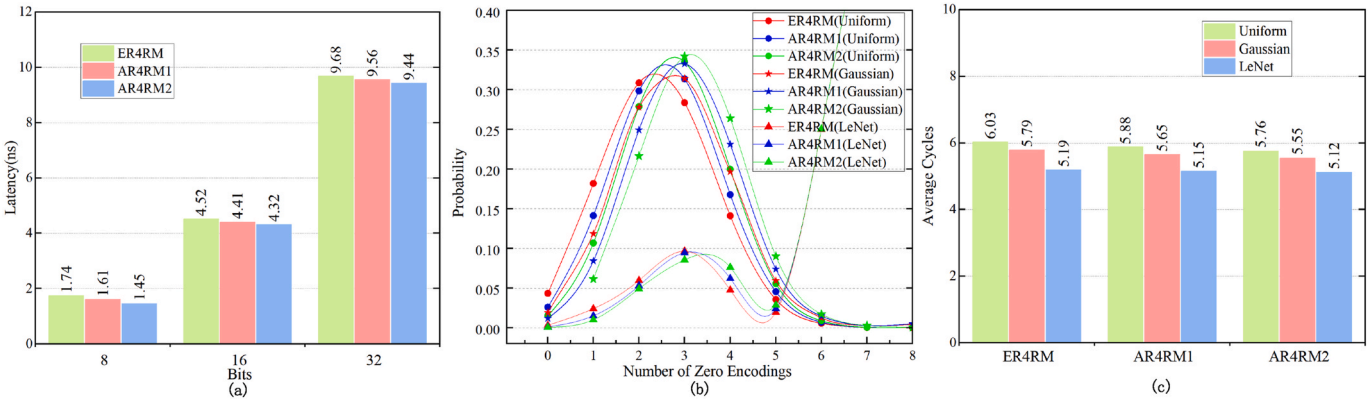

Fig. 7. (a) Latency of the proposed multipliers with different bit widths, (b) Probability distributions of zero encodings in 16-bit multiplier for different sets, (c) Average Cycles of three multipliers in different sets.

Tables 2
16-bit multiplier performance and accuracy metrics on ASIC

Type	Design	Process	Delay (ns)	Latency[a] (ns)	Area (um^2)	Power (uW)	Energy[b] (pJ)	Energy × Area (pJ × um^2)	NMED (10^{-4})	MRED (%)	PRED (%)
Exact	R4BM	40 nm	1.30	11.70	461.99	73.97	0.87	401.93	–	-	-
	R4BPM [16]	45 nm	1.26	1.26	4004.00	1500.00	1.89	7567.56	–	-	-
	DCM [17]	90 nm	1.64	26.20	1716.96	39.10	1.03	1768.47	–	-	-
	R8ES [18]	28 nm	1.20	1.20	440.70	267.80	0.32	85.70	–	-	-
	ER4RM(proposed)	40 nm	1.33	8.02	497.98	76.84	0.62	308.75	–	-	-
	ER4RM(proposed)	28 nm	0.75	4.52	209.92	54.29	0.25	52.48	–	-	-
	ABM-M1 [23]	65 nm	1.20	1.20	3218.00	1200.0	1.44	4504.32	0.05	0.016	–
	ABM-M2 [23]	65 nm	1.20	1.20	2333.00	800.0	0.96	2239.68	2.66	0.66	–
	HLR-BM [24]	45 nm	0.69	0.69	2651.00	3587.0	2.48	6574.48	0.07	0.02	99.1
	Pro1 [25]	28 nm	0.24	0.24	1893.86	2291.0	0.55	1041.62	0.07	1.32	99.9
Approximate	LOZDAM [26]	28 nm	0.92	0.92	523.32	329.1	0.30	156.99	8.00	0.36	99.8
	Park1 [27]	28 nm	0.61	0.61	981.51	754.6	0.46	451.49	0.04	0.015	99.9
	Park2 [27]	28 nm	0.61	0.61	745.99	568.5	0.35	261.10	0.19	0.19	99.4
	AR4RM1(proposed)	40 nm	1.33	7.82	497.09	77.59	0.61	303.22	0.07	0.013	99.9
	AR4RM2(proposed)	40 nm	1.33	7.66	496.74	77.35	0.59	293.08	1.13	0.17	98.9
	AR4RM1(proposed)	28 nm	0.75	4.41	209.91	54.34	0.24	50.38	0.07	0.013	99.9
	AR4RM2(proposed)	28 nm	0.75	4.32	209.72	54.30	0.23	48.24	1.13	0.17	98.9

[a] Latency is the product of delay and cycles.
[b] Energy is the product of latency and power.

using the exact recoding method. Therefore, it generates two approximate partial products and six exact partial products. After adding the partial products, the least significant 18 bits of the product are the approximate bits. For the AR4RM2 with an approximation length of 4, half of the partial products are approximated, and the least significant 22 bits in the result are the approximate bits. Therefore, the accuracy of AR4RM2 is significantly lower than that of AR4RM1 with an approximate length of 2. The approximate length can be further extended, but the accuracy will decrease. For example, when the approximate length is 6, two-thirds of the partial products are approximate values. Furthermore, when the approximate length is 8, all partial products become approximate values, leading to the largest error.

5. Evaluation and comparison

5.1. Multipliers synthesis

The experimental results for the proposed multiplier were analyzed and compared with those of other multipliers in different fields. The proportion of zero encodings in the digit sets influences the performance of the proposed multiplier. Thus, the multiplier evaluation used uniform and Gaussian sets, which are important in ML applications. A 50 % sparse variant of the neural network weight set LeNet [31] was also used. The weight set was generated as single-precision floating points and converted into integers. The uniform sets are generated by selecting numbers between the maximum and minimum values.

In this paper, The proposed exact and approximate designs use the commonly used 16-bit widths to compare with the traditional radix-4 Booth multiplier R4BM, the exact serial multipliers and parallel multipliers in Refs. [16–18], and the advanced approximate multipliers in Refs. [23–27]. The proposed designs were coded in Verilog HDL and synthesized using the TSMC 28-nm CMOS process at the frequency of 200 Mhz. In addition, we also used the TSMC 40-nm CMOS process for comparison. All designs were synthesized without any additional constraints or optimizations and verified using uniform test sets. The parameters were extracted from the Synopsys Design Compiler tool report and are summarized in Table 2, where the latency is the product of delay and average cycles, and the energy is the product of latency and power. Thus, the energy parameter represents the total power consumption to complete a multiplication calculation.

From Table 2, compared with the traditional radix-4 Booth multiplier R4BM, the latency and energy of the proposed ER4RM decreased by 31.5 % and 28.7 % at 40-nm, respectively, while the area and power only increased by 7.8 % and 3.9 %, respectively, due to the addition of the exact recoding module. The radix-4 Booth parallel multiplier R4BPM in Ref. [16] reduces energy consumption by reducing parallel redundant circuits. Compared with R4BPM, ER4RM has a higher latency due to the multi-cycle and small area characteristics, while the area and power are greatly reduced, and the energy and energy × area are reduced by 67.2 % and 95.9 %, respectively. The serial multiplier DCM in Ref. [17] uses a dual-channel serial calculation to simplify the multiplier area, but it has the problems of complex structure and a large number of cycles. The proposed ER4RM has a simple structure and reduces the number of cycles by skipping zero encodings, so it reduces the delay and area by 69.5 % and 71.0 %, respectively, compared with DCM. The exact radix-8 squarer R8ES in Ref. [18] reduces the number of partial products to decrease the delay and area of the squarer, but it is limited to squaring operations only. In contrast, the proposed multiplier employs a zero-skipping technique to reduce calculation delay and supports a broader range of multiplication operations beyond just squaring. Compared with R8ES, ER4RM achieves reductions of 52.4 %, 79.7 %, 21.9 %, and 38.8 % in area, power, energy, and energy × area, respectively, at 28-nm. All the exact zero encodings in Table 1 can be skipped by the proposed multiplier, thereby significantly reducing latency and energy consumption with only a slight increase in area and power. Therefore, the proposed multiplier is more advantageous in designs that require low energy consumption, such as embedded RISC-V processors.

The latency of the proposed multipliers with different bit widths at 28-nm is shown in Fig. 7(a), the multipliers using approximate recoding have lower latency than the exact recoding multiplier in several typical bit widths. The 16-bit sets with different distributions were encoded using the recoding method, generating 8-bit recoding values, and the probability of different numbers of zero encodings was then calculated. Fig. 7(b) shows the probability distributions of zero encodings for three sets. For the same set, the approximate multipliers AR4RM1 and AR4RM2 have more zero encodings than the exact multiplier ER4RM. The Gaussian set has a slightly higher probability of zero encodings than the uniform set. The LeNet set exhibits higher probabilities of zero encodings when the number of zero encodings is greater than 5, because it is 50 % sparse and thus has a large number of zero encodings.

The average cycles of the proposed multiplier in different sets are shown in Fig. 7(c). Compared with ER4RM, the average cycles of the approximate multiplier AR4RM1 and AR4RM2 are lower, because they can recode more zeros, thus skipping more calculations and reducing the computation cycles. AR4RM2 has a higher probability of zero recoding

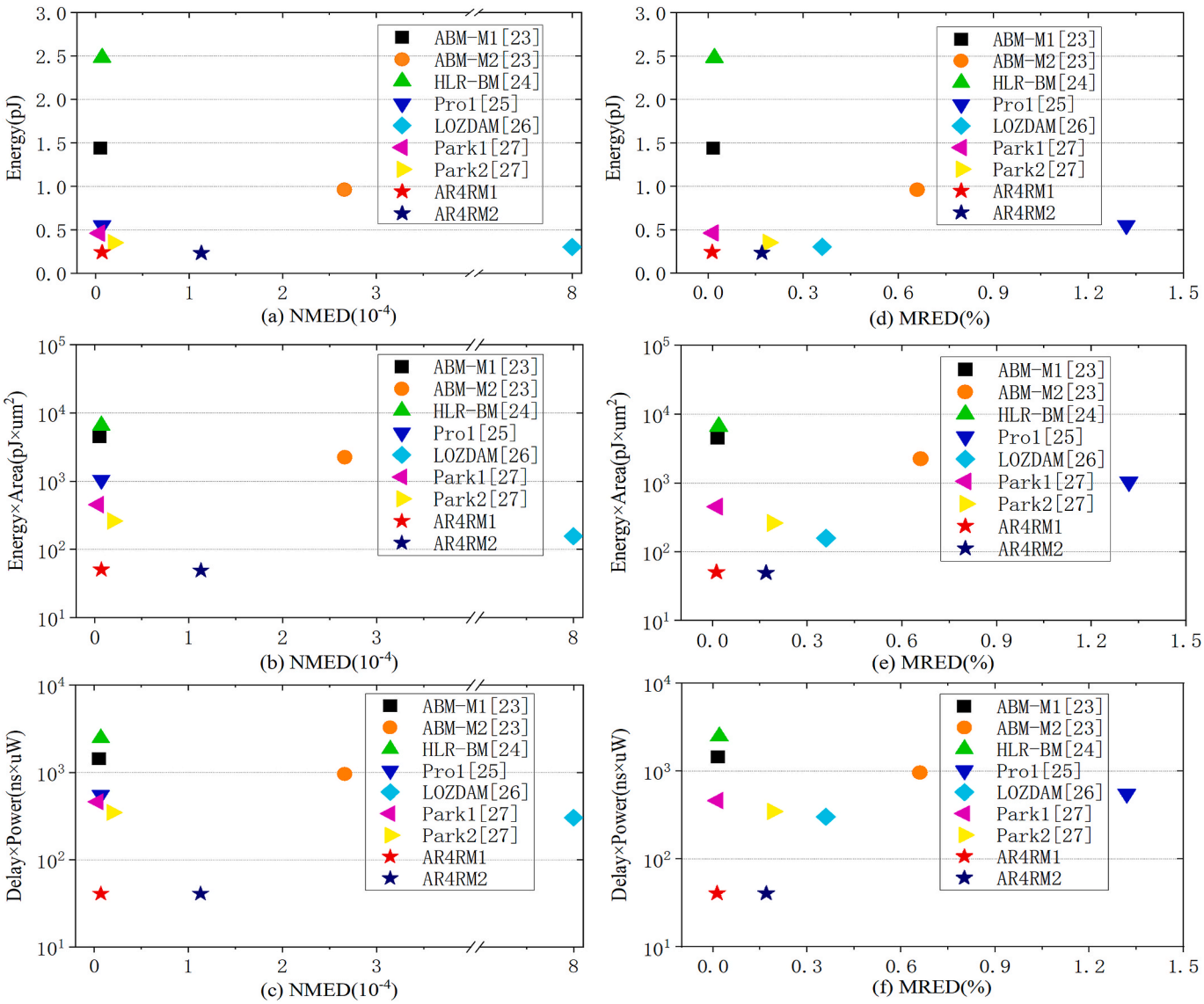

Fig. 8. Hardware and accuracy tradeoff for different 16-bit multipliers. (a) NMED-Energy,(b) NMED-Energy × Area, (c) NMED-Delay × Power, (d) MRED-Energy, (e) MRED-Energy × Area, (f) MRED-Delay × Power.

due to its larger approximate length and more skippable zero encodings, so the average cycles are lower than AR4RM1. For ER4RM, the average cycles of the LeNet set are reduced by 13.9 % and 10.4 % compared to the uniform and Gaussian sets, respectively. Due to the sparse variant of the LeNet set containing 50 % zeros, most computations are "skips", thus the LeNet set has the lowest average cycles among the three sets. Increasing the number of zero encodings and skipping the corresponding calculations can reduce the computation cycle. As shown in Fig. 7(b) and (c), the Gaussian set has more zero encodings than the uniform set, resulting in a 4 % reduction in the average latency. The LeNet set, with significantly more zero encodings than the uniform set, leads to a 13.9 % decrease in average cycles.

5.2. Accuracy analysis

For approximate designs, several metrics are proposed to assess the error characteristic [32,33]. In this paper, M is used to represent the result of the exact multiplier, and M′ is used to represent the result of the approximate multiplier.

Let us indicate as MaxOut the maximum absolute value of the n-bit accurate multiplier: MaxOut $= 2^{2n-2}$ for signed multipliers. The error distance (ED), the normalized mean error distance (NMED) and mean relative error distance (MRED) are defined as follows,

$$ED = |M' - M| \tag{18}$$

$$NMED = \frac{1}{2^{2n} MaxOut} \sum_{i=1}^{2^{2n}} |ED_i| \tag{19}$$

$$MRED = \frac{1}{2^{2n}} \sum_{i=1}^{2^{2n}} \frac{|ED_i|}{|M_i|} \tag{20}$$

Mi represents the exact multiplication result, and EDi represents the error value between the exact and approximate results. The probability of relative error distance(PRED) is the probability of obtaining a RED smaller than a specific percentage value that is assumed to be 2 % throughout this paper. To evaluate the accuracy of the approximate multiplier, the NMED, MRED, and PRED were calculated using MATLAB simulations by one million uniformly distributed random pairs of 16-bit signed inputs ranging from −32768 to 32767.

As the number of zero encodings increases, the accuracy of the multiplier decreases. Table 2 shows that compared with the exact

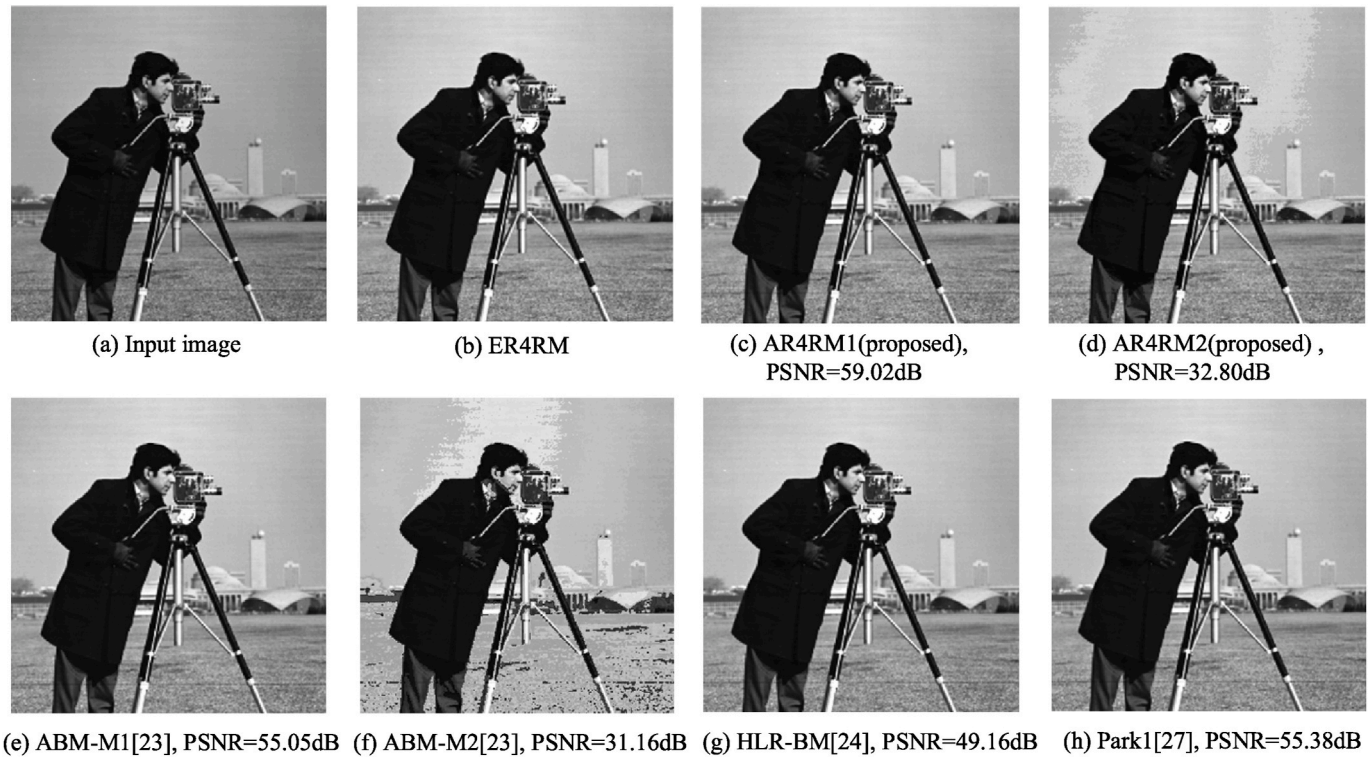

Fig. 9. (a) Input image, Images brightening by using (b) exact multiplier (ER4RM), the approximate multipliers (c) AR4RM1, (d) AR4RM2, (e)ABM-M1[23], (f)ABM-M2[23], (g)HLR-BM[24], (h)Park1[27].

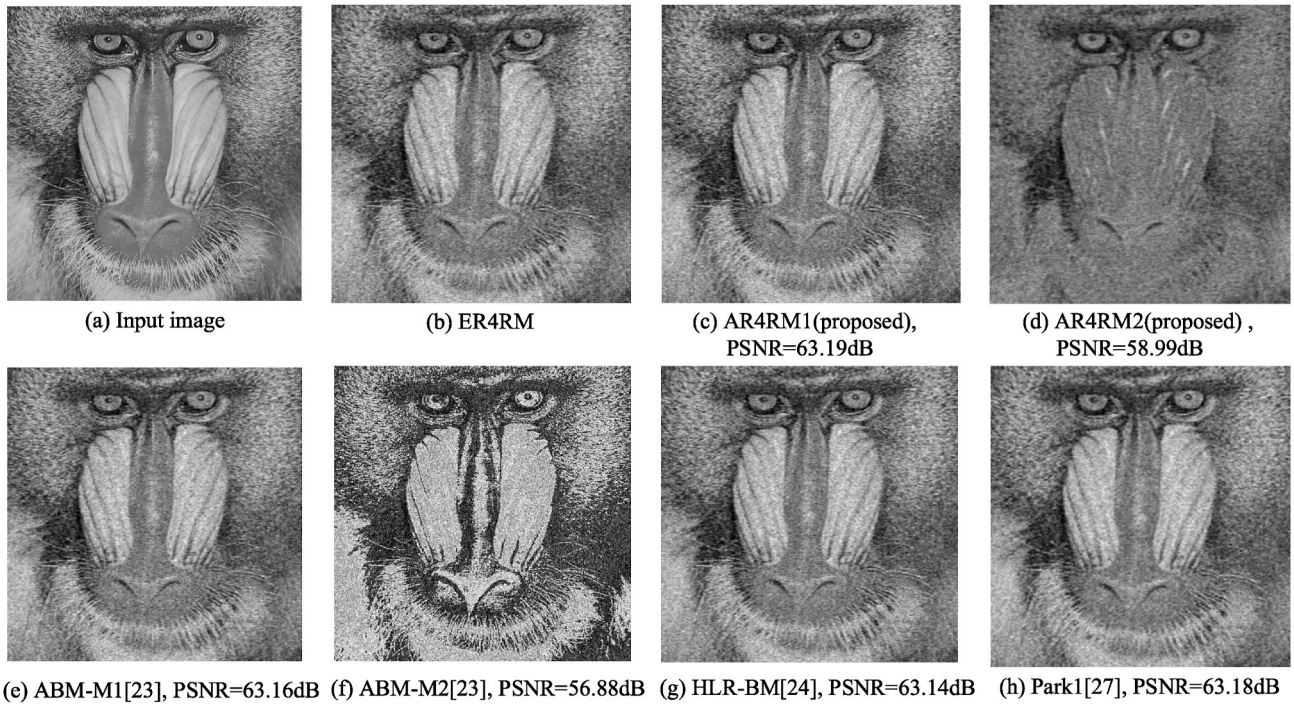

Fig. 10. (a) Input image, Images compression by using (b) exact multiplier (ER4RM), the approximate multipliers (c) AR4RM1, (d) AR4RM2, (e)ABM-M1[23], (f) ABM-M2[23], (g)HLR-BM[24], (h)Park1[27].

multiplier ER4RM, AR4RM1 includes more zero encodings, resulting in a 2.4 % reduction in latency and a 4.0 % reduction in energy at 28-nm, but the NMED and MRED of AR4RM1 increase to 0.07×10^{-4} and 0.013 %, respectively, and PRED is 99.9 %. AR4RM2 has more zero encodings than AR4RM1. Therefore, compared with ER4RM, AR4RM2 reduces the latency and energy by 4.4 % and 8.0 %, respectively, but this also leads to an increase in NMED and MRED to 1.13×10^{-4} and 0.17 %, and PRED is 98.9 %. Therefore, a trade-off between delay and accuracy must be considered in practical applications.

The approximate multiplier ABM-M1 [23] decreases computational

overhead and energy consumption by representing ±2 as ± 1 in radix-4 Booth encoding, and the hybrid low radix multiplier HLR-BM [24] approximates the odd multiples ±3 of radix-8 Booth encoding to ±2 and ± 4, but they do not handle the calculations related to zero encodings. The proposed multiplier enhances efficiency by skipping zero encodings, thereby reducing unnecessary computational overhead. As shown in Table 2, compared with ABM-M1, the energy of the proposed AR4RM1 is reduced by 57.6 % at 40-nm, while maintaining similar accuracy. Compared with HLR-BM, the proposed AR4RM1 reduces area and energy by 81.2 % and 75.4 % at 40-nm, respectively. Pro1 [25] and Park1 [27] improve energy efficiency and accuracy by simplifying the circuit, compared with Pro1 and Park1, AR4RM1 reduces energy by 56.4 % and 47.8 % at 28 nm, and the MRED is reduced by 99.0 % and 13.3 %, respectively. In addition, compared with LOZDAM [26], AR4RM2 reduces energy by 23.3 % at 28 nm, and the NMED and MRED are reduced by 85.9 % and 52.8 %, respectively (see Table 2).

However, the NMED of the proposed design is similar to theirs. Since the proposed multiplier has a serial structure, its area is smaller than other parallel multipliers, resulting in a more significant reduction in energy × area. AR4RM1 achieves 67.9 % lower energy × area than LOZDAM [26] at 28-nm and has significantly lower NMED and MRED. The proposed AR4RM1 has the lowest MRED in Table 2. Each 5-bit approximate recoding of AR4RM1 has an error percentage of only 1/8, an error distance of 1, and only 2 approximate partial products, making it highly accurate. AR4RM2 has a larger approximate length and an approximate partial product of 4, so its NMED and MRED are larger. However, the energy of AR4RM2 is 38.5 %, 23.3 % and 34.3 % lower than that of ABM-M2 [23], LOZDAM [26] and Park2 [27], respectively. In addition, AR4RM1 has a higher PRED than other designs, but AR4RM2 has the lowest PRED since it has more approximate bits.

The trade-off between hardware and NMED tradeoff for different 16-bit multipliers is shown in Fig. 8(a)–(b) and (c). The proposed multipliers AR4RM1 and AR4RM2 show the lowest energy, delay × power and energy × area compared to other state-of-the-art approximate multipliers. In addition, the NMED of AR4RM1 is similar to other state-of-the-art multipliers, but the NMED of AR4RM2 is larger than others and just below ABM-M2, therefore the accuracy of AR4RM2 is lower. The trade-off between hardware and MRED tradeoff for different 16-bit multipliers is shown in Fig. 8(d)–(e) and (f). The proposed multiplier also has the lowest energy, delay × power and energy × area. AR4RM1 has the lowest MRED, and AR4RM2 has less MRED than ABM-M2, Pro1, LOZDAM, Park1 and Park2.

5.3. Image processing

5.3.1. Image brightening

Image brightening is a practical image processing application, therefore, we use it to evaluate the proposed approximate multiplier. The "Cameraman" image is scaled to fit within the range of the 16-bit signed binary digit, so 16-bit approximate signed multipliers can be used to implement the multiplication. Each pixel of the scaled image as input image is multiplied by a predefined constant on its basis to perform image brightening. The results of image transformation using the proposed exact recoding multiplier (ER4RM) and approximate recoding multipliers (AR4RM1 and AR4RM2) are processed using MATLAB. Peak signal-to-noise ratio (PSNR) [34] is used to measure and compare the output image quality for various models. The PSNR metric is defined as follows:

$$PSNR = 10 \times \log_{10}\left(\frac{m \times p \times MAX_I^2}{\sum_{i=0}^{m-1}\sum_{j=0}^{p-1}[I(i,j) - K(i,j)]^2}\right) \quad (21)$$

where, MAX_I is the maximum value of each pixel, m and p are the image dimensions, and $I(i, j)$ and $K(i, j)$ are in turn the exact and obtained values for each pixel.

We take the image brightening result which is processed by an exact multiplier as a reference to calculate the indicators of PSNR. The images produced by the multipliers and their corresponding PSNR values are shown in Fig. 9. Fig. 9(a) shows the original input image, the image by using the accurate multiplier is shown in Fig. 9(b), and the image by using the approximate designs are shown in Fig. 9(c)–(h), respectively. It is obvious that the image processed by ER4RM becomes brighter than the original input image, and there is almost no difference between the images processed by AR4RM1 and ER4RM due to the lower NMED of AR4RM1. However, compared with AR4RM1, the PSNR of AR4RM2 is reduced by 44 %. The image processed by AR4RM2 has lower quality metrics as it has large NMED and MRED, so the processed image has a more obvious change compared with AR4RM1. In addition, compared with ABM-M1, ABM-M2 [23], HLB-BM [24] and Park1 [27], the PSNR of AR4RM1 is improved by 7.2 %, 89.4 %, 20.1 % and 6.6 % respectively.

5.3.2. Image compression

Image compression is also used to test the proposed 16-bit multiplier. The input image "Baboon" is compressed using the Joint Photographic Experts Group, JPEG [35] compression method. The compression process uses the proposed exact and approximate multipliers, the approximate multipliers in Refs. [23,24,27]. The results of image transformation are processed using MATLAB, and the PSNR is calculated according to (20).

The images produced by the multipliers and their corresponding PSNR are shown in Fig. 10. The image compressed using the proposed approximate multiplier AR4RM1 is clearer than that compressed using AR4RM2, as the PSNR of AR4RM1 is 7.1 % higher than that of AR4RM2. Compared with ABM-M1 [23], HLR-BM [24] and Park1 [27], AR4RM1 has a slightly improved PSNR because it has the lowest MRED. In addition, compared with ABM-M2 [23], AR4RM2 has lower NMED and MRED, leading to a 3.7 % improvement in PSNR.

6. Conclusion

In this paper, we proposed novel exact and approximate recoding methods based on the radix-4 Booth encoding for high energy-efficiency designs such as low-power RISC-V processors. The proposed methods convert non-zero encodings to zero encodings in the Booth encoding, therefore, the zero partial product is increased. The exact and approximate radix-4 recoding multipliers were proposed, which detect zero encodings and then skip the zero partial products. In this way, the computation time of the multiplier is reduced, while the energy consumption is also significantly decreased. We designed the exact multiplier ER4RM and two approximate multipliers AR4RM1 and AR4RM2 with different accuracies. They were synthesized using a standard CMOS 28-nm process. Moreover, we investigated 16-bit widths, typical compute sets, such as uniform and Gaussian, and deep neural networks, and evaluated the trade-offs between hardware and accuracy for different designs. Compared with the advanced exact multiplier and approximate multiplier, the results show that the proposed designs have significant advantages in energy and energy × area, with a slight loss of accuracy.

CRediT authorship contribution statement

Xinyu Zhu: Methodology, Writing – original draft. **Hongge Li:** Conceptualization, Project administration, Supervision. **Yinjie Song:** Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported in part by the National Natural Science Foundation of China under Grant (No.U22A6002).

Data availability

Data will be made available on request.

References

[1] B. Moons, M. Verhelst, An energy-efficient precision-scalable ConvNet processor in 40-nm CMOS, IEEE J. Solid-State Circuits 52 (4) (Apr. 2017) 903–914, https://doi.org/10.1109/JSSC.2016.2636225.
[2] S. Bora, R. Paily, A high-performance core micro-architecture based on RISC-V ISA for low power applications, IEEE Trans. Circuits Syst. II, Exp. Briefs 68 (6) (Jun, 2021) 2132–2136, https://doi.org/10.1109/TCSII.2020.3043204.
[3] J. Qian, Y. Ji, C. Li, A runtime-reconfigurable convolutional engine using tensor multiplication with multiple computing modes in 22-nm CMOS, Microelectron. J. 144 (2024), https://doi.org/10.1016/j.mejo.2023.106075.
[4] P. Sun, H. Yu, B. Halak, T. Kazmierski, A method for swift selection of appropriate approximate multipliers for CNN hardware accelerators, in: 2024 *IEEE Int. Symp. On Circuits And Syst*, ISCAS), 2024, pp. 1–5, https://doi.org/10.1109/ISCAS58744.2024.10558159.
[5] E. Antelo, P. Montuschi, A. Nannarelli, Improved 64-bit radix-16 booth multiplier based on partial product array height reduction, IEEE Trans. Circuits Syst. I, Reg. Papers 64 (2) (2017) 409–418, https://doi.org/10.1109/TCSI.2016.2561518. Feb.
[6] D.J.M. Moss, D. Boland, P.H.W. Leong, A two-speed, radix-4, serial–parallel multiplier, IEEE Trans. Very Large Scale Integr. Syst. 27 (4) (Apr. 2019) 769–777, https://doi.org/10.1109/TVLSI.2018.2883645.
[7] N. Amirafshar, A.S. Baroughi, H.S. Shahhoseini, N. TaheriNejad, Carry disregard approximate multipliers, IEEE Trans. Circuits Syst. I, Reg. Papers 70 (12) (Dec. 2023) 4840–4853, https://doi.org/10.1109/TCSI.2023.3306071.
[8] A. Kulkarni, M.A. Ouameur, D. Massicotte, Logic cloning based approximate signed multiplication circuits for FPGA, Microelectron. J. 145 (2024), https://doi.org/10.1016/j.mejo.2024.106135.
[9] A. Booth, A signed binary multiplication technique, Quart. J. Mech. Appl.Math. 4 (1951) 236–240.
[10] O.L. MacSorley, High-speed arithmetic in binary computers, Proc. IRE 49 (1) (1961) 67–91, https://doi.org/10.1109/JRPROC.1961.287779.
[11] L.P. Rubinfield, A proof of the modified Booth's algorithm for multiplication, IEEE Trans. Comput. c-24 (10) (Oct. 1975) 1014–1015, https://doi.org/10.1109/T-C.1975.224114.
[12] U.A. Kumar, S.K. Chatterjee, S.E. Ahmed, Low-power compressor-based approximate multipliers with error correcting module, IEEE Embedded Syst. Lett. 14 (2) (June. 2022) 59–62, https://doi.org/10.1109/LES.2021.3113005.
[13] P.R. Rajput, M.N.S. Swamy, "High speed, efficient area, low power novel modified booth encoder multiplier for signed-unsigned number,'', Adv. Intell. Syst. Comput. 464 (Apr. 2016) 321–333, https://doi.org/10.1007/978-3-319-33625-1_29.
[14] Y. Chang, Y. Cheng, A low power radix-4 booth multiplier with pre-encoded mechanism, IEEE Acess (Jul. 2020) 100–110, https://doi.org/10.1109/ACCESS.2020.3003684.
[15] K. Tsoumanis, N. Axelos, "Pre-Encoded multipliers based on non-redundant radix-4 signed-digit encoding,", IEEE Trans. Comput. 65 (2) (Feb. 2016) 670–676, https://doi.org/10.1109/TC.2015.2428691.
[16] X. Cui, W. Liu, F. Lombardi, A modified partial product generator for redundant binary multipliers, IEEE Trans. Comput. 65 (4) (Apr. 2016) 1165–1171, https://doi.org/10.1109/TC.2015.2441711.
[17] D.M. Ellaithy, M.A. Ei-Moursy, A. Zaki, A. Zekry, Dual-Channel multiplier for piecewise-polynomial function evaluation for low-power 3-D graphics, IEEE Trans. Very Large Scale Integr. Syst. 27 (4) (Apr. 2019) 790–798, https://doi.org/10.1109/TVLSI.2018.2889769.
[18] K. Chen, et al., Exact and approximate squarers for error tolerant applications, IEEE Trans. Comput. 72 (7) (July. 2023) 2120–2126, https://doi.org/10.1109/TC.2022.3228592.
[19] A.G.M. Strollo, E. Napoli, D. De Caro, N. Petra, G. Di Meo, Comparison and extension of approximate 4-2 compressors follow power approximate multipliers, IEEE Trans. Circuits Syst. I, Reg. Papers 67 (9) (Sep. 2020) 3021–3034, https://doi.org/10.1109/TCSI.2020.2988353.
[20] R. Pilipovic, P. Bulic, U. Lotric, A two-stage operand trimming approximate logarithmic multiplier, IEEE Trans. Circuits Syst. I, Reg. Papers 68 (6) (Jun. 2021) 2535–2545, https://doi.org/10.1109/TCSI.2021.3069168.
[21] Z. Aizaz, K. Khare, Area and power efficient truncated booth multipliers using approximate carry-based error compensation, IEEE Trans. Circuits Syst. II, Exp. Briefs 69 (2) (2022) 579–583, https://doi.org/10.1109/TCSII.2021.3094910. Feb.
[22] M.H. Haider, S.B. Ko, Booth encoding-based energy efficient multipliers for deep learning systems, IEEE Trans. Circuits Syst. II, Exp. Briefs 70 (6) (2023) 2241–2245, https://doi.org/10.1109/TCSII.2022.3233923. Jun.
[23] S. Venkatachalam, E. Adams, H.J. Lee, S.B. Ko, Design and analysis of area and power efficient approximate booth multipliers, IEEE Trans. Comput. 68 (11) (Nov. 2019) 1697–1703, https://doi.org/10.1109/TC.2019.2926275.
[24] H. Waris, C. Wang, W. Liu, Hybrid low radix encoding-based approximate booth multipliers, IEEE Trans. Circuits Syst. II, Exp. Briefs 67 (12) (Dec, 2020) 3367–3371, https://doi.org/10.1109/TCSII.2020.2975094.
[25] T. Kong, S. Li, Design and analysis of approximate 4-2 compressors for high accuracy multipliers, IEEE Trans. Very Large Scale Integr. Syst. 29 (10) (Oct. 2021) 1771–1781, https://doi.org/10.1109/TVLSI.2021.3104145.
[26] Y. Du, Z. Chen, B. Cheng, Design and analysis of leading one/zero detector based approximate multipliers, Microelectron. J. 136 (2023), https://doi.org/10.1016/j.mejo.2023.105783.
[27] G. Park, J. Kung, Y. Lee, Simplified compressor and encoder designs for low-cost approximate radix-4 booth multiplier, IEEE Trans. Circuits Syst. II, Exp. Briefs 70 (3) (2023) 1154–1158, https://doi.org/10.1109/TCSII.2022.3217696. Feb.
[28] T. Zhang, H. Jiang, H. Mo, W. Liu, F. Lombardi, L. Liu, J. Han, Design of majority logic-based approximate booth multipliers for error-tolerant applications, IEEE Trans. Nanotechnol. 21 (2022) 81–89, https://doi.org/10.1109/TNANO.2022.3145362.
[29] S.R. Kuang, J.P. Wang, C.Y. Guo, Modified booth multipliers with a regular partial product array, IEEE Trans. Circuits Syst. II, Exp. Briefs 56 (5) (May, 2009) 404–408, https://doi.org/10.1109/TCSII.2009.2019334.
[30] X. Zhu, H. Li, Y. Song, High energy efficiency radix-4 booth multiplier with zero encoding skipping mechanism, in: 2024 IEEE Comput. Society Annual Symp. On VLSI (ISVLSI), 2024.
[31] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet Classification with deep convolutional neural networks,"Proc, 25th Int. Conf. Neural Inf. Process. Syst. (2012) 1097–1105, https://doi.org/10.1145/3065386.
[32] J. Liang, J. Han, F. Lombardi, New metrics for the reliability of approximate and probabilistic adders, IEEE Trans. Comput. 62 (9) (Nov. 2013) 1760–1771, https://doi.org/10.1109/TC.2012.146.
[33] D. Esposito, A.G.M. Strollo, E. Napoli, D. De Caro, N. Petra, Approximate multipliers based on new approximate compressors, IEEE Trans. Circuits Syst. I, Reg. Papers 65 (12) (Dec. 2018) 4169–4182, https://doi.org/10.1109/TCSI.2018.2839266.
[34] Z. Wang, A.C. Bovik, H.R. Sheikh, E.P. Simoncelli, Image quality assessment: from error visibility to structural similarity, IEEE Trans. Image Process. 13 (4) (Apr. 2004) 600–612, https://doi.org/10.1109/TIP.2003.819861.
[35] G.K. Wallace, The JPEG still picture compression standard, IEEE Trans. Consumer Electronics 38 (Feb) (1992).